Optimisers: SGD, Momentum, Adam & AdamW, Worked Out by Hand

Two update steps of each, with bias correction shown

A fully numerical walk-through of how four optimisers turn a gradient into a weight update. We strip away the model and use the simplest possible loss, $L = \frac{1}{2}w^2$, whose gradient is just $g = w$. Starting from $w_0 = 1.0$ with learning rate 0.1, we run two steps of *SGD*, *SGD+momentum*, *Adam* and *AdamW* entirely by hand — every moment, every bias-correction division, every final weight. Nothing is hand-waved: all numbers below were produced by the NumPy/PyTorch reference program in Appendix A and cross-checked against `torch.optim`, so the arithmetic is exact and self-consistent.

What this document does. It takes one scalar weight and watches four update rules move it. Because $g = w$, the gradient at each step is simply the current weight, so you can verify every line in your head. We expose the parts that are usually invisible: the momentum buffer v, Adam's two moments m, v, and the $1 - \beta^t$ bias-correction factors that make the early steps honest.

How to read this. Each optimiser gets its own section with the update rule boxed, then $t = 1$ and $t = 2$ computed explicitly. A *Mini-example* isolates the bias-correction trick; *Intuition* boxes give the one-line mental model; *When-right* and *Watch-out* boxes say which optimiser to reach for and where people get burned (L2-in-the-loss vs. AdamW's decoupled decay).

Concepts covered, in order: the test loss $L = \frac{1}{2}w^2$ · plain SGD · the momentum buffer (EMA of gradients) · Adam's first & second moments · bias correction $m/(1-\beta_1^t)$, $v/(1-\beta_2^t)$ · the adaptive step $\hat{m}/(\sqrt{\hat{v}}+\varepsilon)$ · AdamW's decoupled weight decay · a side-by-side comparison table · an equation cheat-sheet.

1 The setup: one weight, one gradient you can do in your head

An optimiser does not care what the model is; it only sees a parameter and the gradient of the loss with respect to it. So we discard the model entirely and study a single scalar weight w under the smallest non-trivial loss:

$$L(w) \;=\; \tfrac{1}{2}w^2 \qquad \Longrightarrow \qquad g \;=\; \frac{\partial L}{\partial w} \;=\; w.$$

This choice is deliberate: **the gradient at any step equals the current weight**. There is nothing to differentiate by hand — read off w, that *is* g.

Quantity	Symbol	Value here
Initial weight	w_0	1.0
Learning rate	η (`lr`)	0.1
Loss	L	$\frac{1}{2}w^2$
Gradient	g	w
Momentum coefficient	β	0.9
Adam betas	(β_1, β_2)	(0.9, 0.999)
Numerical floor	ε	10^{-8}
AdamW weight decay	λ (`wd`)	0.01

Intuition

Every optimiser in this document is the same skeleton: *look at the gradient, build some running statistic of it, multiply by a step size, subtract from the weight.* SGD uses the raw gradient. Momentum smooths it over time. Adam additionally divides by a running estimate of the gradient's *size*, giving each parameter its own effective learning rate. AdamW then pulls the weight gently toward zero on the side. Hold that ladder in mind — each section adds exactly one rung.

2 Step 1 — SGD: subtract the gradient

Plain stochastic gradient descent has no memory. The update is one line:

$$\boxed{w \;\leftarrow\; w - \eta\, g}$$

With $g = w$, $\eta = 0.1$ and $w_0 = 1.0$:

$$\begin{aligned} t{=}1: &\quad g = w_0 = 1.0, &\quad w_1 &= 1.0 - 0.1 \cdot 1.0 = \boxed{0.900} \\ t{=}2: &\quad g = w_1 = 0.900, &\quad w_2 &= 0.900 - 0.1 \cdot 0.900 = \boxed{0.810} \end{aligned}$$

Each step multiplies the weight by $(1 - \eta) = 0.9$, so the sequence is just 1.0, 0.9, 0.81, … — pure geometric decay toward the minimum at $w = 0$. Simple, but the step size is the same in every direction and never adapts.

3 Step 2 — Momentum: an exponential memory of past gradients

Momentum keeps a *velocity* buffer v that accumulates gradients, then steps along the velocity instead of the raw gradient. Using the course / PyTorch convention (decay on the buffer, gradient added undamped):

$$\boxed{v \leftarrow \beta v + g, \qquad w \leftarrow w - \eta v} \qquad (\beta = 0.9,\ v \text{ starts at } 0).$$

$$t{=}1: \quad g = 1.0, \qquad v = 0.9 \cdot 0 + 1.0 = 1.000, \qquad w_1 = 1.0 - 0.1 \cdot 1.000 = \boxed{0.900}$$

$$t{=}2: \quad g = 0.900, \quad v = 0.9 \cdot 1.0 + 0.900 = 1.800, \quad w_2 = 0.900 - 0.1 \cdot 1.800 = \boxed{0.720}$$

Notice the second step is bigger than SGD's (0.720 vs. 0.810): the velocity $v = 1.8$ carries forward the first gradient, so the optimiser builds speed in a consistently-downhill direction. On this 1-D bowl that means faster descent; on a long narrow valley it is what lets momentum coast through flat stretches and damp the zig-zag across the walls.

Intuition

The buffer v is an (unnormalised) exponential moving average of the gradients: $v_t = g_t + \beta g_{t-1} + \beta^2 g_{t-2} + \cdots$. Recent gradients dominate; old ones fade by a factor β each step. Think of a ball rolling downhill — it does not instantly stop when the slope flattens, because it has accumulated momentum.

4 Step 3 — Adam: two moments, bias-corrected, per-parameter step

Adam keeps *two* running averages. The first moment m is an EMA of the gradient (direction); the second moment v is an EMA of the *squared* gradient (magnitude). Both are true exponential averages, so each new term is weighted by $(1 - \beta)$:

$$m \leftarrow \beta_1 m + (1 - \beta_1)\, g, \qquad v \leftarrow \beta_2 v + (1 - \beta_2)\, g^2.$$

Because m, v both start at 0, they are biased toward zero in the early steps. Adam removes this with the **bias correction** divisors $1 - \beta_1^t$ and $1 - \beta_2^t$, then takes a step whose size is the corrected first moment divided by the (root of the) corrected second moment:

$$\hat{m} = \frac{m}{1 - \beta_1^t}, \qquad \hat{v} = \frac{v}{1 - \beta_2^t}, \qquad \boxed{w \leftarrow w - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon}}$$

Step $t = 1$ ($g = 1.0$; divisors $1 - \beta_1^1 = 0.1$, $1 - \beta_2^1 = 0.001$):

$$m = 0.9 \cdot 0 + 0.1 \cdot 1.0 = 0.100, \qquad v = 0.999 \cdot 0 + 0.001 \cdot 1.0^2 = 0.001,$$

$$\hat{m} = \frac{0.100}{0.1} = 1.000, \qquad \hat{v} = \frac{0.001}{0.001} = 1.000,$$

$$\text{step} = 0.1 \cdot \frac{1.000}{\sqrt{1.000} + 10^{-8}} = 0.100, \quad w_1 = 1.0 - 0.100 = \boxed{0.900}.$$

The bias correction is doing real work here: *before* correction the raw $m = 0.1$ and $\sqrt{v} \approx 0.0316$ would have given the absurd step $0.1 \cdot 0.1 / 0.0316 \approx 0.316$. Dividing m by 0.1 and v by 0.001 rescales

both back to the true gradient scale, and the ratio comes out to exactly 1 — so the first Adam step is $\eta = 0.1$, independent of the gradient's magnitude.

Step $t = 2$ ($g = w_1 = 0.900$; divisors $1 - \beta_1^2 = 0.190$, $1 - \beta_2^2 = 0.002$):

$$m = 0.9 \cdot 0.100 + 0.1 \cdot 0.900 = 0.180, \qquad v = 0.999 \cdot 0.001 + 0.001 \cdot 0.900^2 = 0.001809,$$
$$\hat{m} = \frac{0.180}{0.190} = 0.947, \qquad \hat{v} = \frac{0.001809}{0.002} = 0.905,$$
$$\sqrt{\hat{v}} = 0.951, \qquad \text{step} = 0.1 \cdot \frac{0.947}{0.951 + 10^{-8}} = 0.100,$$
$$w_2 = 0.900 - 0.100 = \boxed{0.800}.$$

(The unrounded step is 0.099588, so $w_2 = 0.800412$, which rounds to 0.800.) Adam's step stays near η because $\hat{m}$ and $\sqrt{\hat{v}}$ track the same gradient and largely cancel — the hallmark of an adaptive, magnitude-normalised method.

Intuition

Adam gives every parameter its *own* effective learning rate $\eta/(\sqrt{\hat{v}} + \varepsilon)$, set by that parameter's recent gradient history. Parameters with large, noisy gradients get $\sqrt{\hat{v}}$ large $\Rightarrow$ a smaller step; quiet parameters get a larger step. The first moment supplies the *direction* (like momentum), the second moment supplies the *scale*. That self-tuning is why Adam works out-of-the-box on wildly different layers without per-layer learning-rate hand-tuning.

Mini-example — this operation in isolation

Why divide by $1 - \beta^t$. A fresh EMA initialised at 0 undershoots. Suppose the gradient is a constant g. After one step $m = (1 - \beta_1)g$, which for $\beta_1 = 0.9$ is only $0.1\,g$ — ten times too small. The total weight assigned so far is exactly $1 - \beta_1^t$ (here $1 - 0.9^1 = 0.1$), so dividing by it renormalises: $\hat{m} = (1 - \beta_1)g/(1 - \beta_1) = g$. As t grows, $\beta^t \to 0$ and the divisor $\to 1$, so the correction quietly switches itself off once the average has warmed up.

Watch out

The $\varepsilon = 10^{-8}$ lives *inside* the denominator, $\sqrt{\hat{v}} + \varepsilon$, not under the root as $\sqrt{\hat{v} + \varepsilon}$. It is a floor that stops a divide-by-zero when a parameter's gradient has been zero for a while ($\hat{v} \to 0$). Putting it in the wrong place, or making it large (10^{-3}), quietly changes the effective step size and is a real cause of "Adam trains differently than expected".

5 Step 4 — AdamW: the same step, plus decoupled weight decay

Weight decay shrinks weights toward zero to regularise. The subtle point AdamW fixes: *where* you apply it. Classic "Adam + L2" folds a term λw into the gradient ($g \leftarrow g + \lambda w$) *before* the moments are computed — so the decay gets run through the $\sqrt{\hat{v}}$ normaliser and is distorted differently for every parameter. AdamW instead **decouples** the decay: do the ordinary Adam step, then subtract $\eta\lambda w$ directly from the weight.

$$\boxed{w \;\leftarrow\; w - \eta\,\frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon} \;-\; \eta\,\lambda\,w} \qquad (\lambda = 0.01).$$

The adaptive part is identical to Section 3 (same $m, v, \hat{m}, \hat{v}$); only the extra decay term is new.

Step $t = 1$ (adaptive step $= 0.100$ as before; decay $= \eta\lambda w_0 = 0.1 \cdot 0.01 \cdot 1.0 = 0.001$):

$$w_1 = 1.0 - 0.100 - 0.001 = \boxed{0.899}.$$

Step $t = 2$ ($g = w_1 = 0.899$; the moments give $\hat{m} = 0.947$, $\hat{v} = 0.904$, adaptive step $= 0.100$; decay $= 0.1 \cdot 0.01 \cdot 0.899 = 0.000899$):

$$w_2 = 0.899 - 0.100 - 0.001 = \boxed{0.799}.$$

(Unrounded: adaptive 0.099582, decay 0.000899, giving $w_2 = 0.798519 \approx 0.799$.) Compared with plain Adam ($w_2 = 0.800$), AdamW lands a hair lower — the decoupled decay pulls every weight slightly toward zero on top of the gradient step, and that small, *uniform* pull is exactly the well-behaved regularisation Adam-with-L2 fails to deliver.

When this is the right choice

AdamW is the default for almost everything — MLPs, Transformers, and most CNNs — because the decoupled decay regularises cleanly and the adaptive step needs little tuning. **SGD + momentum** remains the choice for many vision state-of-the-art results and for fine-tuning pretrained models, where its implicit regularisation and sharper minima can generalise better. Plain **Adam** is fine for quick experiments; switch to AdamW for production.

Watch out

"L2 in the loss" is *not* the same as AdamW's decoupled decay when the optimiser is adaptive. Adding $\frac{\lambda}{2}\|w\|^2$ to the loss injects λw into the gradient, so it passes through Adam's $1/\sqrt{\hat{v}}$ scaling and is effectively weakened for high-variance parameters. AdamW applies $\eta\lambda w$ outside that scaling. If you set `weight_decay` on `torch.optim.Adam` expecting AdamW behaviour, you get the coupled (distorted) version — use `AdamW`.

6 Side by side: where each optimiser put the weight

Optimiser	w after t=1	w after t=2	what carried the step
SGD	0.900	0.810	raw gradient only
SGD + momentum	0.900	0.720	velocity buffer $v = 1.800$
Adam	0.900	0.800	$\hat{m}/(\sqrt{\hat{v}} + \varepsilon)$, adaptive
AdamW	0.899	0.799	adaptive step + decoupled decay

All four agree on w_1 near 0.9 but diverge by $t = 2$: momentum is most aggressive (0.720) because its buffer compounds; SGD is the gentlest of the gradient-only methods (0.810); Adam normalises the magnitude so it sits at 0.800; AdamW shadows Adam a sliver lower (0.799) from the decay. Every value here was reproduced exactly by `torch.optim` (Appendix A).

7 Equation cheat-sheet

SGD	$w \leftarrow w - \eta\, g$
Momentum	$v \leftarrow \beta v + g; \quad w \leftarrow w - \eta\, v$
Adam moments	$m \leftarrow \beta_1 m + (1-\beta_1) g; \quad v \leftarrow \beta_2 v + (1-\beta_2) g^2$
Bias correction	$\hat{m} = m/(1-\beta_1^t); \quad \hat{v} = v/(1-\beta_2^t)$
Adam step	$w \leftarrow w - \eta\, \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$
AdamW step	$w \leftarrow w - \eta\, \hat{m}/(\sqrt{\hat{v}} + \varepsilon) - \eta\, \lambda\, w$
Gradient here	$g = w \quad$ (since $L = \frac{1}{2} w^2$)
Defaults	β_1=0.9, β_2=0.999, ε=10^{-8}

Appendix A Reference implementation

Every number above is reproducible from the following program. The hand computation is written out longhand, then checked against `torch.optim` on the identical loss $L = \frac{1}{2}w^2$ from $w_0 = 1.0$, `lr`$= 0.1$.

```
import numpy as np, torch

w0, lr = 1.0, 0.1                  # L = 0.5*w^2  ->  g = w

# ---- (1) SGD: w <- w - lr*g ----
w = w0
for t in (1, 2):
    g = w
    w = w - lr * g
    print(f"SGD   t={t}: g={g:.3f}  w={w:.3f}")     # 0.900, 0.810

# ---- (2) Momentum (beta=0.9): v <- beta*v + g; w <- w - lr*v ----
beta, w, v = 0.9, w0, 0.0
for t in (1, 2):
    g = w
    v = beta * v + g
    w = w - lr * v
    print(f"MOM   t={t}: g={g:.3f}  v={v:.3f}  w={w:.3f}")  # v=1.0,1.8 ; w=0.900,0.720

# ---- (3) Adam (b1=0.9, b2=0.999, eps=1e-8) ----
b1, b2, eps = 0.9, 0.999, 1e-8
w, m, v = w0, 0.0, 0.0
for t in (1, 2):
    g = w
    m = b1 * m + (1 - b1) * g
    v = b2 * v + (1 - b2) * g * g
    mhat = m / (1 - b1**t)         # bias correction
    vhat = v / (1 - b2**t)
    step = lr * mhat / (np.sqrt(vhat) + eps)   # eps OUTSIDE the sqrt
    w = w - step
    print(f"ADAM  t={t}: mhat={mhat:.3f} vhat={vhat:.3f} step={step:.3f} w={w:.3f}")
# w = 0.900, 0.800

# ---- (4) AdamW: decoupled weight decay (wd applied to w directly) ----
wd = 0.01
w, m, v = w0, 0.0, 0.0
for t in (1, 2):
    g = w
    m = b1 * m + (1 - b1) * g
    v = b2 * v + (1 - b2) * g * g
    mhat, vhat = m / (1 - b1**t), v / (1 - b2**t)
    adapt = lr * mhat / (np.sqrt(vhat) + eps)
    decay = lr * wd * w            # NOT folded into g
    w = w - adapt - decay
    print(f"ADAMW t={t}: adapt={adapt:.3f} decay={decay:.4f} w={w:.3f}")
# w = 0.899, 0.799

# ---- cross-check against torch.optim (float64) ----
def run(make_opt):
    p = torch.nn.Parameter(torch.tensor([1.0], dtype=torch.float64))
    opt = make_opt([p]); out = []
    for _ in range(2):
```

```
        opt.zero_grad(); (0.5 * (p**2).sum()).backward(); opt.step()
        out.append(round(p.item(), 3))
    return out

print("torch SGD  ", run(lambda p: torch.optim.SGD(p, lr=0.1)))                 # [0.9, 0.81]
print("torch MOM  ", run(lambda p: torch.optim.SGD(p, lr=0.1, momentum=0.9)))   # [0.9, 0.72]
print("torch Adam ", run(lambda p: torch.optim.Adam(p, lr=0.1)))                # [0.9, 0.8]
print("torch AdamW", run(lambda p: torch.optim.AdamW(p, lr=0.1, weight_decay=0.01)))  # [0.899,
    0.799]
```

— end of document —